
TIPOS DE DATOS NO LINEALES

SOLUCIÓN:

Ejercicio 1

```
# Explorando el módulo datetime
# a) Importe el módulo datetime y calcule/imprima
# * fecha y hora actual
# * año actual
# * mes del año
# * número de semana del año
# * día de la semana
# * día del año
# * día del mes
```

##Solucion

```
1 import time
2 import datetime
3 print("fecha y hora actual: ", datetime.datetime.now())
4 print("año actual: ", datetime.date.today().strftime("%Y"))
5 print("mes del año: ", datetime.date.today().strftime("%B"))
6 print(" número de semana del año: ", datetime.date.today().strftime("%W"))
7 print("día de la semana: ", datetime.date.today().strftime("%w"))
8 print("día del año: ", datetime.date.today().strftime("%j"))
9 print("día del mes: ", datetime.date.today().strftime("%d"))
```

b) Escriba un programa que calcule la diferencia entre dos fechas

Solución (sencilla)

```
from datetime import date
a = date(2000,2,28)
b = date(2001,2,28)
print(b-a)
```

c) Utilizando los módulos anteriores, escriba un programa que imprima un string cinco veces y que espere tres segundos entre cada print

#Solución

```
import time
x=0
while x<5:
    print("Algoritmos y Estructuras")
    time.sleep(3)
    x=x+1
```

Ejercicio 2:

```
# a) Nos piden que definamos una clase "Persona", la cual tenga como atributos el
nombre,
# el apellido la fecha de nacimiento, la dirección, el teléfono y el email de una persona.
```

```
# b) Además deberemos declarar el método age() dentro de la clase que nos devuelva la edad de la
# persona calculándolo con la fecha actual (pista: debemos importar el módulo 'datetime')
# c) Modifiquemos la clase agregando un atributo edad que se calcule cuando se instancie la clase
# y se actualice cada vez que se llame al método age()
```

##Solución:

#a)

```
import datetime # esto lo usamos para el cálculo de fechas
```

```
class Person:
```

```
    def __init__(self, nombre, apellido, fechaNac, direccion, telefono, email):
        self.nombre = nombre
        self.apellido = apellido
        self.fechaNac = fechaNac

        self.direccion = direccion
        self.telefono = telefono
        self.email = email
```

b)

```
    def age(self):
        hoy = datetime.date.today()
        edad = hoy.year - self.fechaNac.year

        if hoy < datetime.date(hoy.year, self.fechaNac.month, self.fechaNac.day):
            edad -= 1

        return edad
```

c)

```
import datetime # esto lo usamos para el cálculo de fechas
```

```
class Person:
```

```
    ##notatr donde se definió ahora la función age()
```

```
    def age(self):
        hoy = datetime.date.today()
        edad = hoy.year - self.fechaNac.year

        if hoy < datetime.date(hoy.year, self.fechaNac.month, self.fechaNac.day):
            edad -= 1

        return edad
```

```
    def __init__(self, nombre, apellido, fechaNac, direccion, telefono, email):
        self.nombre = nombre
        self.apellido = apellido
        self.fechaNac = fechaNac

        self.direccion = direccion
```

```
self.telefono = telefono
self.email = email
self.edad = self.age()
```

```
## Datos de Ejemplo:
```

```
person = Person(
    "Jane",
    "Doe",
    datetime.date(1992, 3, 12), # year, month, day
    "No. 12 Short Street, Greenville",
    "555 456 0987",
    "jane.doe@example.com"
)
```

```
print(person.nombre)
print(person.email)
print(person.edad)
```

```
## Ejercicio 3
```

```
# a) Crear la clase "Mascota", la misma deberá tener un atributo que sea el nombre de la mascota
```

```
# y una función que imprima un saludo 'de pata'
```

```
# b) A la clase persona agregarle una lista de sus mascotas y un método para agregarle una nueva
```

```
# mascota a la lista (por ahora no mueren ni se eliminan)
```

```
#a)
```

```
class Mascota:
```

```
    def __init__(self, nombre: str):
        self.nombre = nombre
```

```
    def saludoDePata(self):
        print(f"¡{self.nombre} está saludando con las patas!")
```

```
# Datos de Ejemplo
```

```
perro = Mascota('Ayudante de Santa')
```

```
gato = Mascota('Bola de Nieve II')
```

```
# Call a method on one instance
```

```
gato.saludoDePata()
```

```
# Access a property on a class
```

```
print(perro.nombre)
```

```
#b)
```

```
class Person:
```

```
    def __init__(self):
        self.mascotas = [] #debemos inicializar la lista para que no tenga basura
```

```

def agregar_mascota(self, mascota): #método ára agregar mascotas
    self.mascotas.append(mascota)

lisa = Person()
bart = Person()

lisa.agregar_mascota(gato)
bart.agregar_mascota(perro)
print(lisa.mascotas)
print(bart.mascotas)

```

ACLARACIÓN:

Todos tendemos a hacer los siguiente:

```

class Person:
    mascotas = []
    #pero genera error en el polimorfismo de los atributos

```

```

def agregar_mascota(self, mascota):
    self.mascotas.append(mascota)

```

```

lisa = Person()
bart = Person()

```

```

lisa.agregar_mascota(gato)
print(lisa.mascotas)
print(bart.mascotas) # oops! <<--- prueben!!

```

c) Busquen información en la bibliografía o en internet y expliquen que es lo que ocurre en este caso

Ejercicio 4

Explorando colas y pilas

a) Escriba un programa que cree una cola de 4 numeros consecutivos y que imprima todos sus mimebros y su tamaño.

#Solución:

```

import queue
q = queue.Queue()
for x in range(4):
    q.put(x)
print("Miembros de la cola:")
y=z=q.qsize()

```

```

for n in list(q.queue):
    print(n, end=" ")
print("\nTamaño de la cola:")
print(q.qsize())

```